

API Testing



Content

- Introduction To API Testing
- Jest Unit testing
- Supertest Integration Testing
- In-Memory MongoDB For Testing
- Mocking

Introduction To API Testing



API Testing



- **What is Testing in node.js?**
 - Testing in Node.js refers to the process of verifying and validating Node.js applications and their components to ensure they function correctly and meet quality standards.
 - It involves systematically running tests on different parts of the application, such as individual functions, modules, APIs, and overall system interactions, to identify and resolve bugs early in the development cycle.
 - The primary goal of testing in Node.js is to improve code quality, enhance reliability, catch bugs early, and ensure the application behaves as intended, especially when changes or new features are introduced.

API Testing



- **Key aspects of testing your backend app:**

- **Unit Testing:**

This focuses on testing the smallest, isolated units of code, like individual functions or modules, to ensure they produce the expected output for given inputs. Frameworks like Jest, Mocha, and Jasmine are commonly used for unit testing.

- **Integration Testing:**

This involves testing the interactions between different components or modules of the application to ensure they work together seamlessly.

API Testing



- **End-to-End (E2E) Testing:**

This type of testing simulates real user scenarios to verify the entire application flow from start to finish, including interactions with external systems like databases or third-party APIs.

- **Assertion:**

After executing the code under test, assertions are used to compare the actual output or behaviour with the expected results, determining if the test passes or fails. Test frameworks provide assertion libraries to facilitate this process.

API Testing



- **Test Runners:**

These are tools or built-in features (like Node.js's native test runner) that automate the execution of tests and report the results.

- **Test Doubles (Mocks, Stubs, Spies):**

For unit testing, external dependencies (e.g., databases, file system, HTTP services) are often replaced with "fake" implementations (test doubles) to isolate the unit under test and ensure consistent test results.

Unit Testing



API Testing



- **What is Unit Testing? :**
 - Unit testing in Node.js is a software development practice focused on testing individual units or components of a Node.js application in isolation. A "unit" typically refers to the smallest testable part of the code, such as a function, method, or class.
 - The primary goal of unit testing is to verify that each unit of code functions correctly and as expected, independently of other parts of the application or external dependencies like databases, file systems, or external APIs.
 - Popular testing frameworks and libraries used for unit testing in Node.js include Jest, Mocha, Chai, AVA, and Vitest. These tools provide features for defining test suites, writing assertions to verify expected outcomes, and running tests.

API Testing



- **Key characteristics of unit testing:**

- **Isolation:**

Tests are designed to isolate the unit under test, preventing interactions with external dependencies that could introduce variability or complexity. Test doubles (mocks, stubs, spies) are often used to simulate these dependencies.

- **Granularity:**

Focus is placed on testing small, specific functionalities of the code.

API Testing



- **Automation:**

Unit tests are typically automated using specialized testing frameworks and libraries.

- **Speed and Efficiency:**

Unit tests are generally fast to execute, allowing for frequent execution during the development cycle to provide quick feedback.

- **Early Bug Detection:**

By testing individual units, bugs can be identified and fixed early in the development process, reducing the cost and effort of remediation later on.

Jest Unit Testing



- **What is Jest?**
 - Jest is a powerful and easy-to-use JavaScript testing framework developed by Facebook. It is widely used in Node.js and frontend projects alike due to its rich feature set, simplicity, and zero-config setup.
 - Jest is often described as a “batteries-included” testing framework, because it comes bundled with everything you need to write and run tests, including:
 1. A test runner
 2. An assertion library
 3. A mocking utility
 4. Code coverage reports

Jest Unit Testing



- **Why Use Jest in Node.js Projects?**
 - **All-in-One:** No need to install Mocha, Chai, Sinon separately. Jest includes them all.
 - **Zero Configuration:** Works out of the box for most JavaScript and Node.js projects.
 - **Speed:** Parallel test execution makes large test suites faster.
 - **Snapshot Testing:** Great for UI testing (in frontend), but can also be useful in backend for JSON API responses.

Jest Unit Testing



- **Snapshot Testing:** Great for UI testing (in frontend), but can also be useful in backend for JSON API responses.
- **Mocking:** Powerful mocking capabilities for isolating unit tests from external systems (e.g., databases or APIs).
- **Active Ecosystem:** Well-maintained and used by companies like Meta, Airbnb, and Twitter.
- This makes it perfect for beginners and professionals alike who want to get started with testing without configuring multiple tools.

Jest Unit Testing



- How Jest Works:
 1. Jest searches for all files ending in `.test.js` or `.spec.js`.
 2. It executes each test using its built-in test runner.
 3. It uses assertions (e.g., `expect(result).toBe(expected)`) to verify the behavior.
 4. Finally, it prints a report showing:
 - Which tests passed
 - Which failed
 - Execution time
 - Optional: Code coverage percentage

Jest Unit Testing



- How to implement jest in your node.js app :
 - First, start by downloading the module: `$ npm install jest`
 - Then start by configuring jest.config.js:

```
1 export default {  
2   testEnvironment: "node",  
3   transform: {},  
4   extensionsToTreatAsEsm: [".js"]  
5 };
```


Jest Unit Testing



- Then create a test file (e.g. math.test.js) and add the expected behaviour (assertions)

```
1  import { add } from "../services/math.service.js";
2
3  describe("add()", () => {
4    test("adds two positive numbers", () => {
5      expect(add(2, 3)).toBe(5);
6    });
7
8    test("adds negative and positive numbers", () => {
9      expect(add(-2, 5)).toBe(3);
10   });
11
12   test("adds zero", () => {
13     expect(add(0, 0)).toBe(0);
14   });
15
16   test("handles decimal numbers", () => {
17     expect(add(2.5, 1.5)).toBe(4);
18   });
19 });
```

Jest Unit Testing



- In your package.json file, add a script for testing:

```
1 "scripts": {  
2   "test": "node --experimental-vm-modules node_modules/jest/bin/jest.js --coverage",  
3   "start": "node index.js"  
4 },
```

- Simple example function for testing:

```
1 export const add = (a, b) => {  
2   return a + b;  
3 };  
4
```

Jest Unit Testing



- Then simply run `npm test` to start unit testing your functions

```
1  PASS  tests/math.test.js
2    add()
3      ✓ adds two positive numbers (3 ms)
4      ✓ adds negative and positive numbers
5      ✓ adds zero
6      ✓ handles decimal numbers (1 ms)
7
8  -----|-----|-----|-----|-----|-----
9  File           | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
10 -----|-----|-----|-----|-----|-----
11 All files      |      75 |      100 |       50 |      75 |
12  math.service.js |      75 |      100 |       50 |      75 | 6
13 -----|-----|-----|-----|-----|-----
14 Test Suites: 1 passed, 1 total
15 Tests:       4 passed, 4 total
16 Snapshots:   0 total
17 Time:        0.432 s, estimated 1 s
18 Ran all test suites.
```

Integration Testing



Integration Testing



- **What is integration testing? :**
 - Integration testing in Node.js focuses on verifying that different modules or components of an application interact correctly and work together as a cohesive unit.
 - This type of testing is crucial for ensuring the overall functionality and reliability of a Node.js application, especially when dealing with interactions between different parts like routes, services, and databases.
 - It ensures data flows correctly and functionalities work as expected when combined

Integration Testing



- **Key aspects:**
 - **Testing API Endpoints:** A common approach is to test the application's API endpoints, simulating client requests and verifying the responses, including status codes, data format, and content.
 - **Database Interactions:** Integration tests often involve connecting to a real or test database to ensure that the application correctly interacts with the data layer (e.g., creating, reading, updating, and deleting records).
 - **Mocking Dependencies:** While the goal is to test interactions, some external dependencies (like third-party APIs or complex services) might be mocked to isolate the test scope and improve test speed. However, excessive mocking can reduce the "integration" aspect of the test.

Supertest



- **What is Supertest? :**
 - Supertest is a Node.js library used for testing HTTP requests, particularly within the context of testing APIs.
 - It allows developers to write automated tests for their application's routes and endpoints, ensuring they function as expected.
 - Supertest is often used in conjunction with testing frameworks like Jest and Mocha to create a comprehensive testing suite.

Supertest



- **Key features of Supertest:**

- **API Testing:**

Supertest is designed to test APIs, which are essentially the communication pathways between different parts of an application (or different applications).

- **End-to-end Testing:**

It's often used for end-to-end (or integration) testing, ensuring that different parts of the application work together correctly.

Supertest



- **Integration with other frameworks:**

Supertest integrates well with other testing frameworks like Jest, Mocha, and Chai, making it easy to incorporate into existing testing workflows.

- **HTTP Request Testing:**

Supertest provides a fluent API for making HTTP requests (GET, POST, PUT, etc.) against your application and asserting the responses.

- **Automated Testing:**

Supertest is designed for writing automated tests, which are crucial for continuous integration and continuous delivery (CI/CD) pipelines.

Supertest



- **How does Supertest work? :**
 - **You Import Your Express App:**
 - Supertest needs access to your full Express application.
 - You export your app from app.js, but don't start the server.
 - **Supertest Wraps Your App in a Test Client:**
 - Supertest creates a fake client that acts like a browser or Postman.
 - It wraps your Express app using Node's internal HTTP module.
 - It doesn't need to open a port, it talks to your app in memory.

Supertest



- **You Simulate a Request:**
 - You tell Supertest what kind of request to simulate (e.g. POST /login)
 - You can add headers, cookies, or request bodies (e.g., JSON payloads)
- **Supertest Sends the Request Internally:**
 - Supertest sends the fake HTTP request to your Express app exactly as if it came from the internet, but inside the Node process.
 - The request goes through all Express layers: middlewares, routes, controllers.

Supertest



- **Supertest Captures the Response:**
 - It doesn't go back to the browser — instead, Supertest catches it directly.
 - It extracts HTTP status, response headers, JSON body or text
- **You Assert on the Response:**
 - Supertest holds the full response and passes it back to your test.
 - You write assertions to verify:
 - Was the status 200?
 - Did the body contain a token?
 - Did we get an error message?

Supertest



- **Supertest Captures the Response:**
 - It doesn't go back to the browser — instead, Supertest catches it directly.
 - It extracts HTTP status, response headers, JSON body or text
- **You Assert on the Response:**
 - Supertest holds the full response and passes it back to your test.
 - You write assertions to verify:
 - Was the status 200?
 - Did the body contain a token?
 - Did we get an error message?

Supertest



- How to implement Supertest in your express application? :
 - First install the supertest module: `$ npm install supertest`
 - Super test works with jest so do the basic configurations for jest in `jest.config.js`
 - Make sure to export your app to use it in your tests:

```
1 import express from "express";
2 import authRoutes from "../routes/auth.route.js";
3
4 const app = express();
5 app.use(express.json());
6 app.use("/api/auth", authRoutes);
7
8 export default app;
9
```

Supertest



- Finally start by writing your test cases using both jest and supertest:

```
1 import request from "supertest";
2 import app from "../app.js";
3 import { users } from "../utils/users.js";
4
5 beforeEach(() => {
6   users.length = 0;
7 });
8
9 describe("Auth API", () => {
10   test("should register a user", async () => {
11     const res = await request(app)
12       .post("/api/auth/register")
13       .send({ email: "test@example.com", password: "123456" });
14
15     expect(res.statusCode).toBe(201);
16     expect(res.body.message).toBe("User registered");
17   });
18 });
```

In-Memory MongoDB



In-Memory MongoDB



- **What Is In-Memory MongoDB for Testing? :**
 - In-memory MongoDB for testing refers to the practice of using a MongoDB instance that stores its data primarily in a computer's main memory (RAM) rather than on persistent disk storage.
 - This approach is particularly beneficial for testing scenarios due to its speed and ease of setup.
 - in-memory MongoDB provides a fast, isolated, and convenient way to test applications that interact with MongoDB, making it a valuable tool in a developer's testing toolkit.

In-Memory MongoDB



- **Key features for in-memory MongoDB**

- **Speed:**

Data operations (reads and writes) are significantly faster in RAM compared to disk, leading to quicker test execution times.

- **Isolation and Clean State:**

Each test or test suite can spin up a fresh, isolated in-memory MongoDB instance, ensuring that tests don't interfere with each other and always start with a clean database state. This eliminates the need for manual data cleanup between tests.

In-Memory MongoDB



- **Simplified Setup:**

Packages like `mongodb-memory-server` (for Node.js) allow programmatic creation and management of these in-memory instances, removing the dependency on an external, pre-configured MongoDB server.

- **Real MongoDB Behavior:**

While in-memory, these solutions often utilize actual MongoDB binaries, providing a close-to-production environment for integration testing without the overhead of a full-fledged database server.

In-Memory MongoDB



- **Limitations to consider:**

- **Volatility:**

Data stored in memory is lost when the server process is stopped or crashes, making it unsuitable for production environments where data durability is crucial.

- **Limited Feature Support:**

Some advanced MongoDB features, such as transactions or full-text search, may not be fully supported by all in-memory implementations.

In-Memory MongoDB



- How to use in-memory MongoDB?
 - First start install the required modules:

```
$ npm install mongoose mongodb-memory-server
```

- Then in your test suite, create a `beforeAll()` functions that creates the in-memory database server:

```
1  beforeAll(async () => {  
2    mongoServer = await MongoMemoryServer.create();  
3    const uri = mongoServer.getUri();  
4    await mongoose.connect(uri);  
5  });
```

Mocking



Mocking



- **What is mocking?** :
 - Mocking in Jest and Supertest refers to the practice of replacing real dependencies with controlled, simulated versions during testing. This allows for isolated testing of specific code units without relying on external systems or complex setups.
 - Jest has built-in features for creating mock functions and modules.
- **Use mocking for:**
 - Database calls (e.g., Mongoose models)
 - External APIs (e.g., axios, fetch)
 - Utility functions or services

Mocking



Jest mocking functions

- `jest.unstable_mockModule(...)` replaces a module before import.
- `jest.fn()` creates mocks for functions like Express `res.status()`.
- `mockReturnThis()` makes `res.status().json()` chaining possible.
- `mockResolvedValueOnce()` simulates successful DB/API call.
- `toHaveBeenCalledWith(...)` ensures logic is called with correct data.

Mocking



- How to implement mocking in your test suite:

- First mock your service:

```
1  jest.unstable_mockModule('../services/auth.service.js', () => ({  
2    registerUser: jest.fn(),  
3  }));
```

- Then mock your return values from this service:

```
1  const res = {  
2    status: jest.fn().mockReturnThis(),  
3    json: jest.fn(),  
4  };
```

Mocking



- Finally, simulate a successful async function return and add assertions

```
1  authService.registerUser.mockResolvedValueOnce();
2
3      await register(req, res);
4
5      expect(authService.registerUser).toHaveBeenCalledWith('test@example.com', '123456');
6      expect(res.status).toHaveBeenCalledWith(201);
```

- You are not testing whether registerUser() actually creates a user in the database
- You are testing that:
 - The controller calls the service with the correct arguments
 - The controller responds correctly depending on the result of the service

Any Questions?

